

Module 3 Note: Advanced JSX & Patterns

JSX & Elements, Children Types, Cross-Cutting Concerns, HOCs & Render Props

Topic: Web Development / React

Document Version: 1.0

01 JSX Components and Elements

React Elements are plain JS objects describing what should appear on screen (created via `React.createElement` or JSX syntax). **React Components** are functions or classes that return elements.

Types of Children:

- **String / Text Nodes:** Plain textual content nested inside JSX elements.
- **JSX Elements:** Nested component or DOM nodes passed via `props.children`.
- **JavaScript Expressions / Functions:** Functions used as children (Render Props pattern).
- **Booleans, Null, Undefined:** Ignored and unrendered (useful for conditional rendering).

02 Manipulating Children Dynamically in JSX

Because `props.children` can be single elements, arrays, or undefined, direct array manipulation is unsafe. Use the `React.Children` utilities to safely modify children.

```
import React from 'react';

function RadioGroup({ children, selectedValue }) {
  return React.Children.map(children, child => {
    if (!React.isValidElement(child)) return null;
    // Dynamically clone and inject new props into each child
    return React.cloneElement(child, {
      checked: child.props.value === selectedValue
    });
  });
}
```

03 Cross-Cutting Concerns & Cursor Position HOC

Cross-cutting concerns are logic shared across multiple unrelated components (e.g., logging, tracking mouse position, authorization). A **Higher-Order Component (HOC)** is a function that takes a component and returns an enhanced component.

```
import React, { useState, useEffect } from 'react';

// HOC that tracks mouse/cursor position
function withMousePosition(WrappedComponent) {
  return function WithMousePosition(props) {
    const [coords, setCoords] = useState({ x: 0, y: 0 });

    useEffect(() => {
      const handleMouseMove = (e) => {
        setCoords({ x: e.clientX, y: e.clientY });
      };
      window.addEventListener('mousemove', handleMouseMove);
      return () => window.removeEventListener('mousemove', handleMouseMove);
    }, []);

    return <WrappedComponent {...props} mousePosition={coords} />;
  };
}
```

04 Render Props Pattern

The **Render Props** pattern shares stateful logic between components using a prop whose value is a function returning JSX.

```
import React, { useState, useEffect } from 'react';

function MouseTracker({ render }) {
  const [coords, setCoords] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => setCoords({ x: e.clientX, y: e.clientY });
    window.addEventListener('mousemove', handleMouseMove);
    return () => window.removeEventListener('mousemove', handleMouseMove);
  }, []);

  return render(coords);
}

// Usage
function App() {
  return (
    <MouseTracker
      render={({ x, y }) => <p>Cursor at ({x}, {y})</p>
    />
    );
}
```